Анализ вакансий C и C++

Кек Герман Вадимович
ИИИ-3-ИнжДан-1-2, № 91192
Вариант 6

1. Результаты

Отобрано 3 847 вакансий C/C++. Частоты навыков, пар и троек MapReduce и DuckDB полностью совпали для трёх размеров стека. Медиана 893 рублёвых зарплат на руки - 175 000 руб.

Показатель	Значение
Вакансий в исходном CSV	229 388
Вакансий варианта / с навыками	3847 / 2969
Зарплат на руки / до налогов	893 / 411
Зарплата не указана	2497
Предложения в иностранной валюте	46

2. Исходные данные

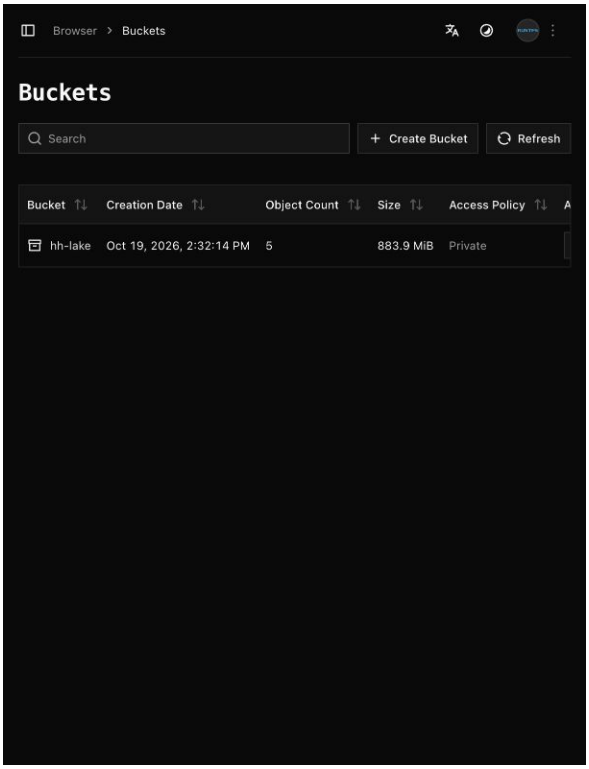
Данные - снимки августа-сентября 2024 года. В data/vacancies.csv сохранены все строки и 13 нужных столбцов основного CSV. Оба расчёта читают этот файл из s3://hh-lake/bronze/vacancies.csv.

Фильтр требует роль «Программист, разработчик» и C/C++ в названии. Учитываются C++, C++, C ++, C/C++ и отдельное C/C/Си. Исключаются C#, Objective-C, C&B, 1 C, 1-C, 1:C и предлог «с». SQL применяет условие самостоятельно; множества ID сверены отдельно.

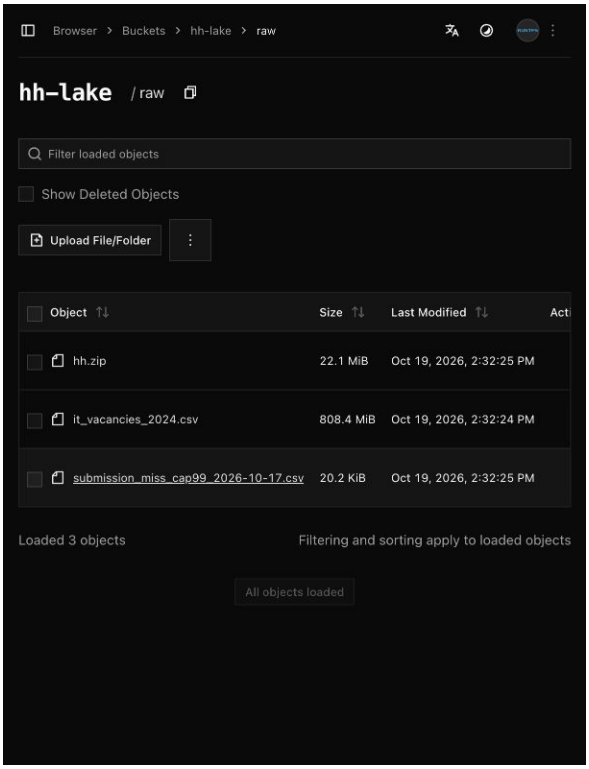
Все 10 000 ID из hh.zip уже есть в основном CSV. Совпали 9 986 названий и 9 997 списков навыков, поэтому повторные снимки не добавлялись. В submission_miss_cap99_2026-10-17.csv есть только mint,tp - этот файл не содержит вакансий. Проверка и SHA-256 файлов: results/data_manifest.json.

1. RustFS и объекты в hh-lake

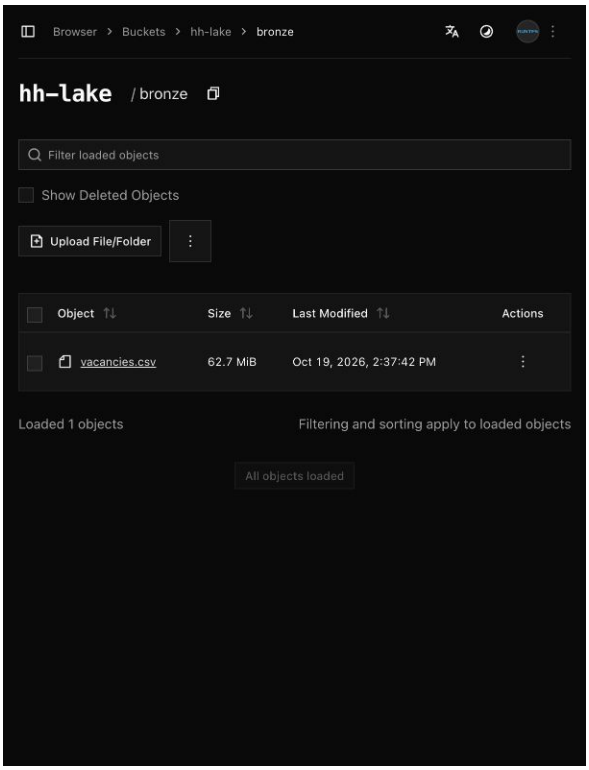
Docker Compose, S3 API - порт 9000, консоль - 9001. Ниже снимки консоли после загрузки данных и результатов.



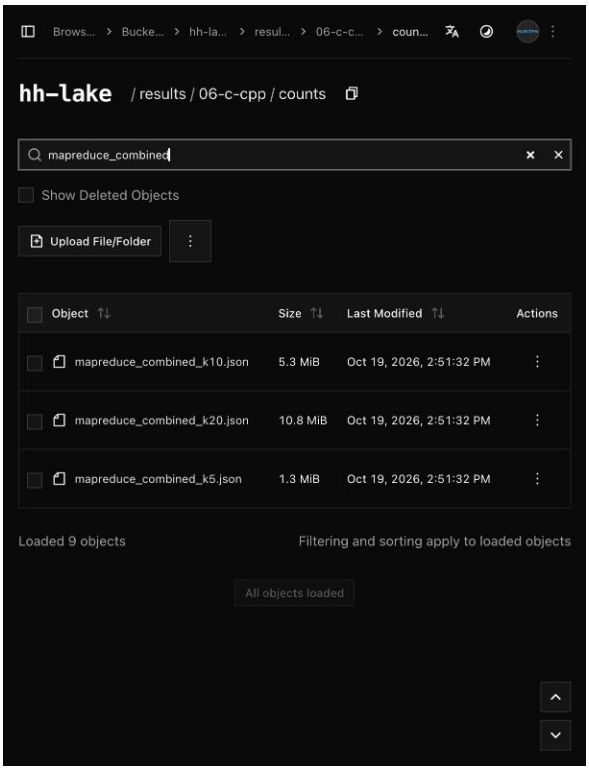
Бакет hh-lake



Исходные файлы в raw/



Общий вход в bronze/



MapReduce: K=5,10,20

```
$ docker compose up -d
$ curl --fail http://127.0.0.1:9000/health
```

{"status": "ok", "service": "rustfs-endpoint", "timestamp": "2026-10-19T14:52:58.693981835Z[Etc/Unknown]", "version": "1.0.1"}

2. Подсчёт и верификация

1. Реализация

- Навыки разделяются по ;, очищаются от пробелов и приводятся к нижнему регистру
- Повторы внутри вакансии удаляются, синонимы не объединяются
- K - первые навыки после сортировки, а не топ-K по популярности

2. Результаты

K	Навыки	Пары	Тройки	Уникальных ключей
5	12 200	21 183	19 184	12 532
10	15 699	43 588	82 867	47 891
20	16 882	58 598	174 312	92 111

3. Обоснование

Вакансия с k навыками даёт k одиночных упоминаний, k(k-1)/2 пар и k(k-1)(k-2)/6 троек. При 5 навыках получается 10 пар и 10 троек, при 20 - 190 пар и 1 140 троек. Поэтому рост стека сильнее влияет на тройки.

MapReduce читает S3 потоком, чанками по 2 000 строк. Два процесса выполняют map, родитель складывает счётчики; в очереди не больше четырёх задач. DuckDB самостоятельно читает тот же S3-объект через httpfs и строит сочетания по индексам i < j < k.

4. Проверка

Сравнены все ключи и частоты, размер выборки и суммы упоминаний. SHA-256 полных словарей:

K=5: 229e8cc72976b336f4d7d12a768758f7f9ce01d9e562b5413da96a19b5259759

K=10: 3269cb7dba8c3b4c4cf88a6dc154f69eeed7ee087a5a6dce57fcea8adcb8e6f2

K=20: 39ff3de5389bcf33af6acdb491cae5ca7c768d980941a566ac6c60dbccfbd803

```
$ hh verify
K=5 совпало
K=10 совпало
K=20 совпало
```

Контрольный отказ задачи № 0 вызвал один повтор. Совпадение после восстановления: True. Reducer принимает только целиком завершённую задачу, поэтому частоты не удваиваются. Проверено исключение задачи, не аварийная гибель процесса ОС.

Девять файлов частот сохранены в results/06-c-cpp/counts/ бакета hh-lake. Три результата MapReduce прочитаны обратно из S3 и сверены. Локальные доказательства: verification.json и failure_test.json.

3. Время, память и оптимизация

1. Методика

- Три попытки на конфигурацию после одного прогрева движка
- Порядок движков меняется, кэши ОС и RustFS не сбрасываются
- Время включает запуск Python, S3, расчёт и запись JSON; загрузка результатов в S3 выполняется после замера

2. Результаты

Движок	K	Медиана, с	Min / max, с	σ, с	RSS median / max, MiB
baseline	5	1.692	1.626 / 1.819	0.098	234.4 / 240.0
combined	5	1.565	1.556 / 1.636	0.044	167.4 / 168.0
duckdb	5	1.194	1.176 / 1.366	0.105	276.6 / 310.3
baseline	10	1.802	1.766 / 1.811	0.024	239.0 / 243.5
combined	10	1.738	1.723 / 1.760	0.019	175.8 / 177.7
duckdb	10	1.487	1.477 / 1.597	0.066	307.9 / 357.0
baseline	20	2.000	1.975 / 2.092	0.062	246.3 / 251.1
combined	20	1.975	1.964 / 2.004	0.020	214.9 / 215.9
duckdb	20	1.821	1.805 / 1.990	0.103	332.2 / 426.0

3. Оптимизация MapReduce

K	Время до / после, с	Снижение	RSS до / после, MiB	Снижение
5	1.692 / 1.565	7.5%	234.4 / 167.4	28.6%
10	1.802 / 1.738	3.5%	239.0 / 175.8	26.4%
20	2.000 / 1.975	1.3%	246.3 / 214.9	12.8%

Оптимизация фильтрует и сокращает строки до передачи процессам и использует локальный combiner. Она экономит память; малый выигрыш по времени нужно сопоставлять с разбросом попыток. DuckDB быстрее в этих запусках, но хранит промежуточные таблицы и потребляет больше RSS.

Память - пик суммы RSS родителя и потомков, опрос раз в 10 мс. Общие страницы могут учитываться повторно, короткие пики - пропускаться. Два процесса MapReduce / два потока DuckDB, лимит DuckDB 1 GB. Среда: macOS-26.6.2-arm64-arm-64bit, 8 CPU, 16.0 GiB RAM. RustFS измерен отдельно: hh-rustfs | 0.08% | 272.3MiB / 7.661GiB.

4. Сочетания с более высокой зарплатой

1. Методика

Зарплата - середина вилки или единственная указанная граница. Основная когорта: положительные конечные суммы RUR/RUB на руки (gross=false). До налогов и валюты учитываются отдельно, пропуски не заменяются нулями. В анализе используется полный стек.

Для пары база - более частый одиночный навык, для тройки - более частая входящая пара. Частота базы выбирается по всем вакансиям, медиана - по сопоставимым зарплатам. Нужны минимум 20 зарплат в сочетании и 20 у базы без добавленного навыка. Рейтинг - по разнице в рублях.

2. Результаты

Сочетание	База	n / n базы	Медиана / база, руб	Разница, руб	%
c++ + postgresql	c++	47 / 418	225 000 / 175 000	50 000	28.6
qt + математическое программирование	qt	24 / 178	185 000 / 150 000	35 000	23.3
postgresql + sql	sql	29 / 76	205 000 / 175 000	30 000	17.1
c++ + stl	c++	62 / 418	200 000 / 175 000	25 000	14.3
linux + sql	linux	47 / 365	205 000 / 180 000	25 000	13.9
c++ + qt + ооп	c++ + qt	35 / 141	200 000 / 150 000	50 000	33.3
bash + git + linux	git + linux	23 / 153	250 000 / 200 000	50 000	25.0
linux + qt + ооп	linux + qt	34 / 97	187 500 / 140 000	47 500	33.9
c++ + linux + postgresql	c++ + linux	38 / 231	225 000 / 180 000	45 000	25.0
git + linux + python	git + linux	32 / 153	237 500 / 200 000	37 500	18.8

3. Вывод

C++ + PostgreSQL: 225 тыс. против 175 тыс. у C++ (+50 тыс., n=47). C++ + Qt + ООП: 200 тыс. против 150 тыс. у C++ + Qt (+50 тыс., n=35). Эти различия могут отражать уровень вакансий и состав работодателей.

5. Сочетания с более низкой зарплатой

1. Результаты

Те же правила базы и поддержки. n - число зарплат на руки, а не всех вакансий с сочетанием.

Сочетание	База	n / n базы	Медиана / база, руб	Разница, руб	%
linux + opengl	linux	20 / 365	102 500 / 180 000	-77 500	-43.1
c# + c++	c++	24 / 418	100 000 / 175 000	-75 000	-42.9
bash + python	python	21 / 85	105 000 / 165 000	-60 000	-36.4
c++ + opengl	c++	33 / 418	125 000 / 175 000	-50 000	-28.6
bash + git	git	32 / 223	140 000 / 190 000	-50 000	-26.3
c/c++ + linux + разработка по	c/c++ + linux	20 / 150	102 500 / 182 500	-80 000	-43.8
bash + c/c++ + linux	c/c++ + linux	21 / 150	105 000 / 182 500	-77 500	-42.5
c/c++ + linux + ооп	c/c++ + linux	30 / 150	120 000 / 182 500	-62 500	-34.2
c++ + c/c++ + ооп	c++ + ооп	22 / 74	130 000 / 190 000	-60 000	-31.6
c++ + c/c++ + linux	c++ + linux	44 / 231	127 500 / 180 000	-52 500	-29.2

2. Вывод и ограничения

Linux + OpenGL: 102,5 тыс. против 180 тыс. у Linux (-77,5 тыс., n=20). C# + C++: 100 тыс. против 175 тыс. у C++ (-75 тыс., n=24). Группы небольшие: возможны различия в опыте, задачах и работодателях. Это не означает, что сами навыки снижают зарплату.

Для Bash + Git общая разница равна -50 тыс. руб., а внутри одинаковых региона и уровня опыта знак становится положительным. Поэтому по общей медиане нельзя делать причинный вывод.

6. Сравнение регионов

1. Результаты

В численность входят все вакансии варианта, включая записи без навыков. Зарплатные показатели рассчитаны по рублям на руки.

Регион	Все / с навыками	n net / покрытие	Медиана / среднее, руб	Q25 / Q75, руб
Москва	1762 / 1334	348 / 19.8%	200 000 / 219 200	160 000 / 250 000
Санкт-Петербург	751 / 590	182 / 24.2%	155 000 / 176 382	130 000 / 205 000
Новосибирск	178 / 144	56 / 31.5%	150 000 / 166 250	107 500 / 221 250

Регион	Навыки	Пары	Тройки	Средний стек
Москва	7 467	25 414	72 448	4.24
Санкт-Петербург	3 225	10 343	28 338	4.29
Новосибирск	737	2 476	8 739	4.14

2. Вывод

В Москве медиана 200 000 руб., в Петербурге - 155 000 руб., в Новосибирске - 150 000 руб. Москва выше Петербурга на 29.0%, Новосибирска - на 33.3%.

Покрытие зарплатами ниже в Москве (19.8%), чем в Новосибирске (31.5%). Сравнение относится к опубликованным предложениям. Квартильные интервалы пересекаются, поэтому город не определяет зарплату сам по себе.

Число троек превышает число пар из-за длинных стеков отдельных вакансий. Региональные численности, медианы и частоты дополнительно сверены отдельным Python-проходом из S3. Полные таблицы - regions_summary.csv и regions_skills.csv.

7. Навыки в Москве и Санкт-Петербурге

Топ-3 навыков, пар и троек каждого региона. Доля считается от всех вакансий региона. Полный топ-10 по каждому порядку содержит 90 строк.

Москва

Тип	Навык или сочетание	Частота	% всех	n net	Медиана, руб
Навык	c++	833	47.3	173	225 000
Навык	linux	704	40.0	149	225 000
Навык	c/c++	382	21.7	100	200 000
Пара	c++ + linux	398	22.6	98	225 000
Пара	c/c++ + linux	254	14.4	57	225 000
Пара	git + linux	244	13.8	55	225 000
Тройка	c++ + git + linux	136	7.7	34	212 500
Тройка	c++ + linux + qt	107	6.1	31	200 000
Тройка	c/c++ + git + linux	82	4.7	21	210 000

Санкт-Петербург

Тип	Навык или сочетание	Частота	% всех	n net	Медиана, руб
Навык	c++	369	49.1	75	150 000
Навык	linux	304	40.5	80	150 000
Навык	c/c++	188	25.0	51	170 000
Пара	c++ + linux	173	23.0	45	160 000
Пара	c/c++ + linux	122	16.2	29	150 000
Пара	git + linux	109	14.5	33	170 000
Тройка	c++ + git + linux	55	7.3	17	200 000
Тройка	c++ + linux + qt	55	7.3	21	140 000
Тройка	c/c++ + git + linux	47	6.3	12	150 000

Пара C++ + Linux лидирует в обоих городах: 398 и 173 вакансии. Её медиана на руки - 225 тыс. в Москве и 160 тыс. в Петербурге. Похожие стеки не устраняют разницу регионального состава вакансий.

8. Новосибирск и проверка зарплатных выводов

Новосибирск

Тип	Навык или сочетание	Частота	% всех	n net	Медиана, руб
Навык	c++	104	58.4	37	107 500
Навык	linux	68	38.2	20	190 000
Навык	c/c++	42	23.6	15	165 000
Пара	c++ + linux	35	19.7	9	180 000
Пара	c/c++ + linux	33	18.5	10	200 000
Пара	git + linux	32	18.0	11	200 000
Тройка	git + linux + tcp/ip	16	9.0	4	180 000
Тройка	c++ + git + linux	15	8.4	6	180 000
Тройка	c/c++ + git + linux	15	8.4	5	250 000

В Новосибирске лидирует тройка Git + Linux + TCP/IP (16 вакансий), в Москве - C++ + Git + Linux (136). Это различие публикуемых стеков, а не доказательство специализации всего города. Для редких сочетаний нужно смотреть n зарплат рядом с медианой.

Проверка по региону и опыту

Сочетание	Общая разница, руб	По стратам, руб	Страт
c++ + postgresql	50 000	-15 000	2
bash + git	-50 000	25 000	1
bash + git + linux	50 000	25 000	1
c++ + linux + postgresql	45 000	-28 000	2

Для C++ + PostgreSQL +50 тыс. меняются на -15 тыс. внутри сопоставимых страт. Нужны минимум 5 зарплат с навыком и без него в одной группе «регион × опыт»; веса - меньшая из двух численностей. Подходящих страт мало, поэтому это проверка чувствительности.

В salary_effects.csv также сохранены разницы с непересекающейся базой и 95% bootstrap-интервалы (1 000 повторов). Интервалы не скорректированы за множественный поиск. Независимо проверены 20 сочетаний и все 90 строк региональных навыков. Ограничение зарплат 10 тыс. - 2 млн руб. не изменило медиану: 175 000 руб.

9. Запуски профилирования

27 попыток

Каждая строка - отдельный процесс, код выхода 0. Все словари совпали с эталоном своего K. Прогревы не включены.

Попытка	K	Движок	Время, с	Пик RSS, MiB
1	5	baseline	1.626	234.4
1	5	combined	1.556	166.8
1	5	duckdb	1.194	310.3
1	10	baseline	1.766	243.5
1	10	combined	1.723	174.8
1	10	duckdb	1.477	357.0
1	20	baseline	2.000	246.3
1	20	combined	2.004	214.9
1	20	duckdb	1.990	305.2
2	5	combined	1.565	168.0
2	5	duckdb	1.176	276.6
2	5	baseline	1.692	240.0
2	10	combined	1.738	175.8
2	10	duckdb	1.487	292.5
2	10	baseline	1.802	239.0
2	20	combined	1.975	204.1
2	20	duckdb	1.821	332.2
2	20	baseline	2.092	246.3
3	5	duckdb	1.366	274.0
3	5	baseline	1.819	233.1
3	5	combined	1.636	167.4
3	10	duckdb	1.597	307.9
3	10	baseline	1.811	238.9
3	10	combined	1.760	177.7
3	20	duckdb	1.805	426.0
3	20	baseline	1.975	251.1
3	20	combined	1.964	215.9

Исходные замеры: results/benchmark_runs.csv. Выводы запусков: results/logs/. Проверка инфраструктуры: results/commands.txt. Команды повторения: hh load и hh run.